Wapiti vulnerability report

Target: https://google-gruyere.appspot.com/4856106547737856153166721281125965970 41/

Date of the scan: Wed, 29 Oct 2025 13:53:48 +0000. Scope of the scan: folder. Crawled pages: 10

Summary

Category	Number of vulnerabilities found
Backup file	0
Cleartext Submission of Password	0
Weak credentials	0
CRLF Injection	0
Content Security Policy Configuration	1
Cross Site Request Forgery	0
Potentially dangerous file	0
Command execution	0
Path Traversal	0
Fingerprint web application framework	0
Fingerprint web server	0
Htaccess Bypass	0
HTML Injection	0
Clickjacking Protection	1
HTTP Strict Transport Security (HSTS)	1
MIME Type Confusion	1
HttpOnly Flag cookie	1
Unencrypted Channels	0

Category	Number of vulnerabilities found
Inconsistent Redirection	0
Information Disclosure - Full Path	0
LDAP Injection	0
Log4Shell	0
NS takeover	0
Open Redirect	0
Reflected Cross Site Scripting	1
Secure Flag cookie	1
Spring4Shell	0
SQL Injection	0
TLS/SSL misconfigurations	0
Server Side Request Forgery	0
Stored HTML Injection	0
Stored Cross Site Scripting	9
Subdomain takeover	0
Blind SQL Injection	0
Unrestricted File Upload	0
Vulnerable software	0
Internal Server Error	0
Resource consumption	0
Review Webserver Metafiles for Information Leakage	0
Fingerprint web technology	0
HTTP Methods	0

Category	Number of vulnerabilities found
TLS/SSL misconfigurations	0

Content Security Policy Configuration

Description

Content Security Policy (CSP) is an added layer of security that helps to detect and mitigate certain types of attacks, including Cross Site Scripting (XSS) and data injection attacks.

Vulnerability found in /485610654773785615316672128112596597041/

Description | HTTP Request | cURL command line | WSTG Code

```
CSP is not set for URL: https://google-gruyere.appspot.com/485610654773785615316672128112596597041/
```

Solutions

Configuring Content Security Policy involves adding the Content-Security-Policy HTTP header to a web page and giving it values to control what resources the user agent is allowed to load for that page.

References

- Mozilla: Content Security Policy (CSP)
- OWASP: Content Security Policy Cheat Sheet
- OWASP: How to do Content Security Policy (PDF)
- OWASP: Content Security Policy

Clickjacking Protection

Description

Clickjacking is a technique that tricks a user into clicking something different from what the user perceives, potentially revealing confidential information or taking control of their computer.

Vulnerability found in /485610654773785615316672128112596597041/

Description | HTTP Request | cURL command line | WSTG Code

```
X-Frame-Options is not set
```

Solutions
Implement X-Frame-Options or Content Security Policy (CSP) frame-ancestors directive.

References

- OWASP: Clickjacking
- KeyCDN: Preventing Clickjacking

HTTP Strict Transport Security (HSTS)

Description
HSTS is a web security policy mechanism that helps to protect websites against man-in-the-middle attacks such as protocol downgrade attacks and cookie hijacking.

🟡 Vulnerability found in /4856106547737856153166721281125965970 41/

Description | HTTP Request | cURL command line | WSTG Code

```
Strict-Transport-Security is not set
```

Solutions
Implement the HTTP Strict Transport Security header to enforce secure connections to the server.

References

- OWASP: HTTP Strict Transport Security
- KeyCDN: Enabling HSTS

MIME Type Confusion

Description
MIME type confusion can occur when a browser interprets files as a different type than intended, which could lead to security vulnerabilities like cross-site scripting (XSS).

🟡 Vulnerability found in /4856106547737856153166721281125965970 41/

Description | HTTP Request | cURL command line | WSTG Code

```
X-Content-Type-Options is not set
```

Solutions

Implement X-Content-Type-Options to prevent MIME type sniffing.

References

- OWASP: MIME Sniffing
- KeyCDN: Preventing MIME Type Sniffing

HttpOnly Flag cookie

Description

HttpOnly is an additional flag included in a Set-Cookie HTTP response header. Using the HttpOnly flag when generating a cookie helps mitigate the risk of client side script accessing the protected cookie (if the browser supports it).

🟡 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
GET /4856106547737856153166721281125965970 41/saveprofile?
action=new&uid=default&pw=Letm3in_&is_author=True HTTP/1.1
host: google-gruyere.appspot.com
connection: keep-alive
user-agent: Mozilla/5.0 (Windows NT 6.1; rv:45.0) Gecko/20100101 Firefox/45.0
accept-language: en-US
accept-encoding: gzip, deflate, br
accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/webp,*/*;q=0.8
```

Solutions

While creation of the cookie, make sure to set the HttpOnly Flag to True.

References

- OWASP: Testing for Cookies Attributes
- OWASP: HttpOnly

Reflected Cross Site Scripting

Description

Cross-site scripting (XSS) is a type of computer security vulnerability typically found in web applications which allow code injection by malicious web users into the web pages viewed by other users. Examples of such code include HTML code and client-side scripts.

🟠 Vulnerability found in /4856106547737856153166721281125965970 41/snippets.gtl

Description | HTTP Request | cURL command line | WSTG Code

```
Reflected Cross Site Scripting vulnerability found via injection in the parameter uid
```

Solutions

The best way to protect a web application from XSS attacks is ensure that the application performs validation of all headers, cookies, query strings, form fields, and hidden fields. Encoding user supplied output in the server side can also defeat XSS vulnerabilities by preventing inserted scripts from being transmitted to users in an executable form. Applications can gain significant protection from javascript based attacks by converting the following characters in all generated output to the appropriate HTML entity encoding: <, >, &, ', (,), #, %, ; , +, -

References

- OWASP: Cross Site Scripting (XSS)
- Wikipedia: Cross-site scripting
- CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
- OWASP: Reflected Cross Site Scripting

Secure Flag cookie

Description

The secure flag is an option that can be set by the application server when sending a new cookie to the user within an HTTP Response. The purpose of the secure flag is to prevent cookies from being observed by unauthorized parties due to the transmission of a the cookie in clear text.

🟡 Vulnerability found in /4856106547737856153166721281125965970 41/

saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Secure flag is not set on the cookie: 'GRUYERE' set at 'https://google-gruyere.appspot.com/4856106547
```

Solutions

When generating the cookie, make sure to set the Secure Flag to True.

References

- OWASP: Testing for Cookies Attributes
- OWASP: Secure Cookie Attribute

Stored Cross Site Scripting

Description

Cross-site scripting (XSS) is a type of computer security vulnerability typically found in web applications which allow code injection by malicious web users into the web pages viewed by other users. Examples of such code include HTML code and client-side scripts.

🔴 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

🔴 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

🔴 Vulnerability found in /485610654773785615316672128112596597041/saveprofile

Description HTTP Request cURL command line WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

🔴 Vulnerability found in /485610654773785615316672128112596597041/saveprofile

Description HTTP Request cURL command line WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

🔴 Vulnerability found in /485610654773785615316672128112596597041/saveprofile

Description HTTP Request cURL command line WSTG Code

```
Stored Cross Site Scripting vulnerability found via injection in the parameter uid
```

🔴 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Stored Cross Site Scripting vulnerability found via injection in the parameter uid
```

🔴 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

🔴 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

🔴 Vulnerability found in /4856106547737856153166721281125965970 41/saveprofile

Description | HTTP Request | cURL command line | WSTG Code

```
Stored Cross Site Scripting vulnerability found in https://google-gruyere.appspot.com/485610654773785
```

Solutions

The best way to protect a web application from XSS attacks is ensure that the application performs validation of all headers, cookies, query strings, form fields, and hidden fields. Encoding user supplied output in the server side can also defeat XSS vulnerabilities by preventing inserted scripts from being transmitted to users in an executable form. Applications can gain significant protection from javascript based attacks by converting the following characters in all generated output to the appropriate HTML entity encoding: <, >, &, ', (,), #, %, ; , +, -

References

- OWASP: Cross Site Scripting (XSS)
- Wikipedia: Cross-site scripting
- CWE-79: Improper Neutralization of Input During Web Page Generation ('Cross-site Scripting')
- OWASP: Stored Cross Site Scripting
